

rSLDS Description: NNT Lab

Bhavya Surapaneni

August 4th, 2026

Abstract

This document aims to explain the methodology and details of applications of the Recurrent Switching Linear Dynamical Systems (rSLDS) [1] model, as it is applied to datasets in the Purdue NanoNeurotechnology Lab.

Overview rSLDS is a dynamical systems modeling paradigm that utilizes both switching dynamics and recurrence that identifies and uniquely generates latent dynamics of a system. This model builds on former work with plain Switching Linear Dynamical Systems, or SLDS. In SLDS, a standard Markov Chain determines the switching between discrete states; in rSLDS, a latent continuous state is used to track switches between discrete states, with feedback where the continuous state influences future discrete switches—this is the introduction of recurrence.

rSLDS is uniquely useful for neuroscience dynamics modeling for the following reasons:

1. It is a dynamical systems paradigm, which means that the results are entirely driven by math and we can go deeper into the intricacies of each discrete state, represented as a linear dynamical system. This is a major benefit over black-box machine learning methods.
2. The dynamics that rSLDS represents are locally linear (each discrete state is a linear system), but the latent continuous state allows global nonlinearity as a function of movement through the linear discrete states.
3. The model is highly adaptable - it can be used for EPhys data, Calcium data, or other forms of data, as long as you set an emissions type that aligns with your data structure. Most simply, this would be Poisson Orthogonal emissions for spiking/ephys, and Gaussian Orthogonal for calcium and most 'analog' data structures, though there are various other options built into the library that can be explored.

Some other important things to note about the model include:

1. The model runs on a single session, so model results between sessions are not correlated. This means that discrete states are individual to each session and to correlate them, you either need to do more statistical analysis or just behaviorally correlate them. I've mostly focused on behavioral correlation.

2. Data must be loaded as an array of the structure `(num_timesteps, num_neurons)`. Binning and smoothing is highly recommended for electrophysiology data, and tends to substantially improve results for calcium imaging data as well.

Model Details When running the model there are certain hyperparameters that must be set. These are manually set by the user but depend on the data dimensionality and structure. Thus, the best way to go about determining these values is to run cross-validation on the dataset and test various values to see how much variance is explained by certain parameter settings.

For most datasets in this lab, including Shivam, Shulan's, and DJ's data, I chose to run the model with **4 discrete states and 8 continuous dimensions**. This was a decision made from running cross-validation on a set of Shulan's data and finding that 4 states, 8 dimensions pretty consistently explained 50-60% of the variance. Cross-validation is a very lengthy, computationally intensive process so I stuck with these hyperparameters for different datasets.

The convenient caveat is that if you run the model with 4 discrete states, if the model only identifies 3 major states, the resultant flow fields, trajectories, and most-likely-state array will not force a 4th state—they will simply identify the 3 states. Thus, there is no harm in running the model with the extra state, as it offers a nice buffer in case there are more complex dynamics to be found in the data.

The continuous dimensions that you select are important as they define the dimensionality of your resultant connectivity (A) matrix. Each discrete state has a connectivity matrix, and it will be of dimension `(num_continuous_dims x num_continuous_dims)`. So, in our case, every connectivity matrix is an (8,8) matrix, and these connectivity matrices are what determine the generated dynamics.

Pipeline Structure I have included the following diagram, which shows a highly simplified pipeline of what the model does.

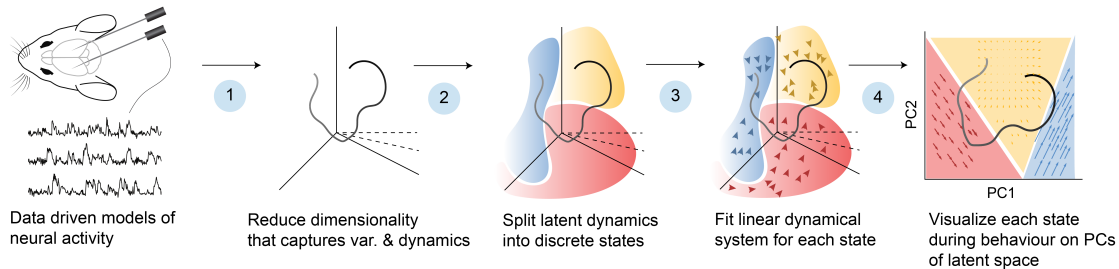


Figure 1: rSLDS process diagram

This is essentially just a visual of what I described in the overview. The only notable point is

that when I plot the flow fields and trajectories, I plot them on the **axes of the first and second principal components of the latent space** - specifically, the latent dimensions.

To do this, we run PCA on the latent dimensions, which are the resultant 8 dimensions that the model found to represent the data's hidden state space, or the 'space' the latent dynamics exist within. By doing this, we make sure that the flow field and trajectory we plot are plotted on the axes that explain the most variance from the state space. This allows us to run the model with a high number of dimensions, allowing a more complex state space, but still be able to visualize the results in 2D.

State-Specific Analysis State-specific analysis can be done since each state has its own connectivity matrix. The eigenvalues from these matrices are especially useful, as they can be used for further calculations.

You can identify the time constants of a state using an equation from Nair et al. 2022 [2]. The equation is as follows:

$$\tau = \left| \frac{1}{\log(|\lambda_a|)} \right|$$

Nair uses the time constants to identify an 'integration dimension', which is the dimension that corresponds to the longest time constant and is meant to be the spatial dimension along which integration occurs.

You can also find a line attractor score for the entire discrete state as per the following:

$$score = \log_2 \frac{t_n}{t_{n-1}}$$

where t_n is the largest time constant and t_{n-1} the second largest [2]. Line attractors are indicative of integration, so this can give insight into functionally what is happening as per your dynamics.

Hammad has also developed some useful analysis techniques for individual states. You can identify **single neuron contributions** from neuron weights, which the model returns as a C matrix, to see whether the dynamics are generated by many neurons or a small, sparser set. You can also identify **spiral attractor dynamics** using a spiral score:

$$score = w \times \frac{|\lambda|}{|Im(\lambda)|}$$

where w is a growth weight value.

A spiral score can be useful to identify the length of dynamics, as a spiral may indicate a longer time period to return to a stable state after perturbation (stimulus).

Source Code All pipeline code is stored in this GitHub repository: <https://github.com/bhavyasur/state-space-neuro.git>. The original model source code is in the SSM library on GitHub. This library should be imported from GitHub rather than from PyPI, as the PyPI version is not updated and has some bugs.

I have done my best to keep the code structure as simple as possible and will update the `README.md` to explain everything necessary. Please forward any questions about the repository to **bhavyasp@engineering.upenn.edu**

References

- [1] Scott W. Linderman, Andrew C. Miller, Ryan P. Adams, David M. Blei, Liam Paninski, and Matthew J. Johnson. Recurrent switching linear dynamical systems, 2016.
- [2] Aditya Nair, Tomomi Karigo, Bin Yang, Scott W Linderman, David J Anderson, and Ann Kennedy. An approximate line attractor in the hypothalamus that encodes an aggressive internal state. *bioRxiv*, 2022.